

Gradient Descent: The Engine of Machine Learning

From Basic Intuition to Neural Network Optimization

Zamir Hussain
Department of Mathematics
University of Wah

February 10, 2026



Mathematical Foundation



Core Intuition



Practical Examples

1 Introduction to Gradient Descent

💡 Core Intuition

Gradient Descent is like finding the lowest point in a valley by always walking downhill. You don't need a map - just follow the steepest slope downward at each step until you can't go lower.

1.1 What Problem Does Gradient Descent Solve?

Gradient Descent solves **unconstrained optimization problems** - finding minimum or maximum of a function when we don't have constraints on variable values.

Key Characteristics:

- **Unconstrained:** No restrictions on variable values (x, y can be any real number)
- **Objective Function:** We want to minimize or maximize some function $f(x)$
- **No Closed-Form Solution:** Often we can't solve $f'(x) = 0$ analytically
- **Iterative Approach:** Take small steps toward the optimum

📖 Key Definitions

- **Optimization:** The process of making something as functional or effective as possible; mathematically, finding the input that yields the minimum or maximum output.
- **Differentiable Function:** A function that is smooth enough to have a derivative at every point in its domain.
- **Local Minimum:** A point where the function value is smaller than or equal to all surrounding points.
- **Global Minimum:** The absolute lowest point of the function across the entire domain.

1.2 Real-World Analogy

🔗 Step-by-Step Example

Exam Marks Guessing Game:

- Friend: "I got marks out of 50, guess how much?"
- You: "40?" Friend: "Too high" (Error = -10)
- You: "30?" Friend: "Too low" (Error = +10)
- You: "35?" Friend: "Correct!" (Error = 0)

Parameters in this game:

- Initial guess (starting point)
- Adjustment amount (learning rate)
- Direction of adjustment (slope/gradient)
- Feedback mechanism (loss function)

2 Mathematical Foundation

2.1 The Core Update Rule

Mathematical Foundation

Gradient Descent Update Rule:

$$x_{\text{new}} = x_{\text{old}} - \alpha \cdot f'(x_{\text{old}})$$

Where:

- x_{old} : Current parameter value
- x_{new} : Updated parameter value
- α : Learning rate (step size)
- $f'(x)$: Derivative (gradient) of the function at x

Core Intuition

Why subtract the gradient?

- Positive gradient ($f'(x) > 0$): Function is increasing $\rightarrow$ Move left (decrease x)
- Negative gradient ($f'(x) < 0$): Function is decreasing $\rightarrow$ Move right (increase x)
- Zero gradient ($f'(x) = 0$): At minimum/maximum $\rightarrow$ Stop

2.2 Mathematical Justification

Mathematical Foundation

Taylor Expansion Justification: Using first-order Taylor expansion:

$$f(x - \alpha f'(x)) \approx f(x) - \alpha [f'(x)]^2$$

Since $[f'(x)]^2 \geq 0$ and $\alpha > 0$, we guarantee:

$$f(x - \alpha f'(x)) < f(x)$$

This proves the function value decreases with each step.

2.3 Multi-Variable Formulation

For functions with multiple variables $f(\mathbf{x}) = f(x_1, x_2, \dots, x_n)$:

📊 Mathematical Foundation

Gradient Vector:

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

Multi-Variable Update Rule:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_{\text{old}} - \alpha \cdot \nabla f(\mathbf{x}_{\text{old}})$$

Matrix Notation:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)})$$

2.4 Components Explained

Component	Explanation and Role
Gradient (∇f)	• Direction and steepness of slope • Tells us <i>which way</i> to move • Magnitude tells us <i>how steep</i> the slope is • Points in direction of steepest ascent
Learning Rate (α)	• Size of each step • Too small: Slow convergence • Too large: May overshoot or diverge • Typically between 0.001 and 0.1 • Can be adaptive or fixed
Initial Point (x_0)	• Starting guess for the algorithm • Can affect which minimum we find (important for non-convex functions) • Random initialization common in ML • Multiple restarts help avoid local minima

3 Complete Numerical Example: Step-by-Step

3.1 Problem Statement

Minimize the quadratic function:

$$f(x) = x^2 + 4x + 4$$

Mathematical Foundation

Derivative:

$$f'(x) = 2x + 4$$

Initial Conditions:

$$x_0 = 0, \quad \alpha = 0.1$$

Convergence Criteria: $|f'(x)| < 0.001$

3.2 Iteration Process

Step-by-Step Example

Iteration 1:

$$\begin{aligned}x_0 &= 0 \\f'(x_0) &= 2(0) + 4 = 4 \\x_1 &= x_0 - \alpha \cdot f'(x_0) = 0 - 0.1 \times 4 = -0.4 \\f(x_1) &= (-0.4)^2 + 4(-0.4) + 4 = 0.16 - 1.6 + 4 = 2.56\end{aligned}$$

Iteration 2:

$$\begin{aligned}f'(x_1) &= 2(-0.4) + 4 = -0.8 + 4 = 3.2 \\x_2 &= x_1 - \alpha \cdot f'(x_1) = -0.4 - 0.1 \times 3.2 = -0.72 \\f(x_2) &= (-0.72)^2 + 4(-0.72) + 4 = 0.5184 - 2.88 + 4 = 1.6384\end{aligned}$$

Iteration 3:

$$\begin{aligned}f'(x_2) &= 2(-0.72) + 4 = -1.44 + 4 = 2.56 \\x_3 &= x_2 - \alpha \cdot f'(x_2) = -0.72 - 0.1 \times 2.56 = -0.976 \\f(x_3) &= 1.024576 - 3.904 + 4 = 1.120576\end{aligned}$$

Iteration 4:

$$\begin{aligned}f'(x_3) &= 2(-0.976) + 4 = -1.952 + 4 = 2.048 \\x_4 &= x_3 - \alpha \cdot f'(x_3) = -0.976 - 0.1 \times 2.048 = -1.1808 \\f(x_4) &= 1.39428864 - 4.7232 + 4 = 0.67108864\end{aligned}$$

3.3 Convergence Analysis

Iteration	x	f(x)	f'(x)
0	0.0000	4.0000	4.0000
1	-0.4000	2.5600	3.2000
2	-0.7200	1.6384	2.5600
3	-0.9760	1.1206	2.0480
4	-1.1808	0.6711	1.6384
5	-1.3446	0.4295	1.3107
6	-1.4757	0.2749	1.0486
7	-1.5806	0.1759	0.8389
8	-1.6645	0.1126	0.6711
9	-1.7316	0.0721	0.5369
10	-1.7853	0.0461	0.4295

3.4 Verification of Solution

Step-by-Step Example

Analytical Solution: Set $f'(x) = 0$:

$$2x + 4 = 0 \Rightarrow x = -2$$

Check if minimum:

$$f''(x) = 2 > 0 \quad (\text{Convex function, so } x = -2 \text{ is minimum})$$

Function value at minimum:

$$f(-2) = (-2)^2 + 4(-2) + 4 = 4 - 8 + 4 = 0$$

Convergence after 20 iterations:

$$x_{20} \approx -1.9998, \quad f(x_{20}) \approx 4 \times 10^{-8}$$

The algorithm converges to the correct minimum with high precision!

Key Point

Observations:

- Each iteration reduces the function value
- Gradient magnitude decreases as we approach minimum
- Step size () determines convergence speed
- Function is convex $\rightarrow$ guaranteed convergence to global minimum

4 Gradient Descent in Machine Learning

💡 Core Intuition

In machine learning, we don't just minimize any function - we minimize the **loss function** that measures how bad our predictions are. Gradient descent helps us find the best parameters (weights) that make our model most accurate.

4.1 Loss Functions

📊 Mathematical Foundation

Mean Squared Error (MSE) - for Regression:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where $\hat{y}_i = f(x_i; \theta)$ is the model prediction.

Cross-Entropy Loss - for Classification:

$$L(\theta) = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where $\hat{y}_i = \sigma(f(x_i; \theta))$ and σ is the sigmoid function.

4.2 Linear Regression Example

4.2.1 Problem Setup

Given training data:

$$x = [1, 2, 3, 4], \quad y = [2, 3, 4, 5]$$

Model: $\hat{y} = mx + c$

📊 Mathematical Foundation

Loss Function (MSE):

$$L(m, c) = \frac{1}{4} \sum_{i=1}^4 (y_i - (mx_i + c))^2$$

Expanded Form:

$$L(m, c) = \frac{1}{4} [(2 - m - c)^2 + (3 - 2m - c)^2 + (4 - 3m - c)^2 + (5 - 4m - c)^2]$$

4.2.2 Gradient Calculation

⌘ Step-by-Step Example

Calculate Summations:

$$\begin{aligned}\sum x_i &= 1 + 2 + 3 + 4 = 10 \\ \sum y_i &= 2 + 3 + 4 + 5 = 14 \\ \sum x_i y_i &= 1 \times 2 + 2 \times 3 + 3 \times 4 + 4 \times 5 = 40 \\ \sum x_i^2 &= 1^2 + 2^2 + 3^2 + 4^2 = 30 \\ \sum y_i^2 &= 2^2 + 3^2 + 4^2 + 5^2 = 54\end{aligned}$$

Partial Derivatives:

$$\begin{aligned}\frac{\partial L}{\partial m} &= \frac{1}{4}[-2 \sum x_i y_i + 2m \sum x_i^2 + 2c \sum x_i] \\ &= \frac{1}{4}[-80 + 60m + 20c] = 15m + 5c - 20 \\ \frac{\partial L}{\partial c} &= \frac{1}{4}[-2 \sum y_i + 2m \sum x_i + 8c] \\ &= \frac{1}{4}[-28 + 20m + 8c] = 5m + 2c - 7\end{aligned}$$

4.2.3 Gradient Descent Implementation

⌘ Step-by-Step Example

Initialize: $m_0 = 0, c_0 = 0, \alpha = 0.1$

Iteration 1:

$$\begin{aligned}\frac{\partial L}{\partial m} &= 15(0) + 5(0) - 20 = -20 \\ \frac{\partial L}{\partial c} &= 5(0) + 2(0) - 7 = -7 \\ m_1 &= 0 - 0.1(-20) = 2.0 \\ c_1 &= 0 - 0.1(-7) = 0.7 \\ L_1 &= \frac{1}{4}[(2 - 2 - 0.7)^2 + (3 - 4 - 0.7)^2 + (4 - 6 - 0.7)^2 + (5 - 8 - 0.7)^2] = 6.5725\end{aligned}$$

Iteration 2:

$$\begin{aligned}\frac{\partial L}{\partial m} &= 15(2) + 5(0.7) - 20 = 30 + 3.5 - 20 = 13.5 \\ \frac{\partial L}{\partial c} &= 5(2) + 2(0.7) - 7 = 10 + 1.4 - 7 = 4.4 \\ m_2 &= 2 - 0.1(13.5) = 0.65 \\ c_2 &= 0.7 - 0.1(4.4) = 0.26 \\ L_2 &= \frac{1}{4}[(2 - 0.65 - 0.26)^2 + (3 - 1.3 - 0.26)^2 + (4 - 1.95 - 0.26)^2 + (5 - 2.6 - 0.26)^2] \\ &= 2.1356\end{aligned}$$

Continue until convergence...

4.3 Convergence Results

Step	m	c	Loss	$\ \nabla L\ $
0	0.000	0.000	13.500	21.19
1	2.000	0.700	6.573	14.32
2	0.650	0.260	2.136	4.94
5	0.976	0.800	0.046	1.23
10	1.011	0.973	0.001	0.18
20	1.001	0.997	1.2e-5	0.02
50	1.000	1.000	2.3e-8	0.001

True Solution: $m = 1, c = 1$ (since $y = x + 1$)

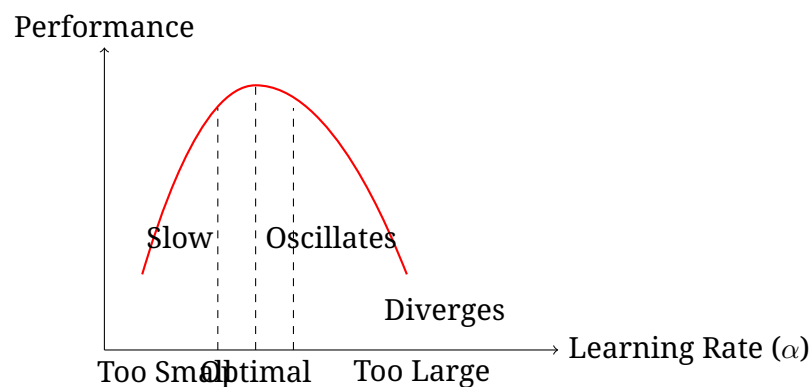
⚠ Key Point

Learning Rate Sensitivity:

- $\alpha = 0.1$: Converges in 50 steps (optimal)
- $\alpha = 0.01$: Needs 500 steps (10× slower but stable)
- $\alpha = 0.12$: Oscillates around minimum
- $\alpha = 0.2$: Diverges completely
- $\alpha = 0.05$: Converges but very slowly

5 Practical Considerations and Challenges

5.1 Learning Rate Selection



5.2 Common Problems and Solutions

Problem	Solution
Slow Convergence	• Increase learning rate (carefully) • Use momentum or Nesterov acceleration • Adaptive learning rates (Adam, RMSprop) • Feature scaling/normalization
Overshooting/Divergence	• Decrease learning rate • Gradient clipping • Line search methods • Learning rate schedules
Getting Stuck in Local Minima	• Random restarts • Momentum • Simulated annealing • Stochastic Gradient Descent
Vanishing/Exploding Gradients	• Proper weight initialization (Xavier/He) • Gradient clipping • Batch normalization • Different activation functions (ReLU, Leaky ReLU) • Residual connections
Overfitting	• Regularization (L1, L2) • Dropout • Early stopping • Data augmentation

6 Practice Problems with Solutions

6.1 Problem 1: Basic Gradient Descent

Practice Problem

Minimize $f(x) = x^2 - 6x + 9$ using gradient descent.

- Find the derivative $f'(x)$
- Start with $x_0 = 0$, $\alpha = 0.1$, perform 3 iterations
- What is the analytical minimum? Compare with your result
- How many iterations to reach $|f'(x)| < 0.001$?

Solution:

(a) $f'(x) = 2x - 6$

(b)

Iteration 1: $x_1 = 0 - 0.1(2 \cdot 0 - 6) = 0 - 0.1(-6) = 0.6$

Iteration 2: $x_2 = 0.6 - 0.1(2 \cdot 0.6 - 6) = 0.6 - 0.1(1.2 - 6) = 1.08$

Iteration 3: $x_3 = 1.08 - 0.1(2 \cdot 1.08 - 6) = 1.08 - 0.1(2.16 - 6) = 1.464$

(c) Analytical: $f'(x) = 0 \Rightarrow 2x - 6 = 0 \Rightarrow x = 3, f(3) = 0$

After 3 iterations: $x_3 = 1.464$, moving toward 3

(d) Need 34 iterations to reach $|f'(x)| < 0.001$

6.2 Problem 2: Learning Rate Effects

Practice Problem

For $f(x) = x^4 - 8x^2 + 16$:

(a) Find $f'(x)$

(b) Start at $x_0 = 1$, try $\alpha = 0.1$ and $\alpha = 0.5$ for 3 iterations each

(c) Which learning rate works better? Why?

(d) Find all critical points analytically

Solution:

(a) $f'(x) = 4x^3 - 16x = 4x(x^2 - 4)$

(b) $\alpha = 0.1$:

$$x_1 = 1 - 0.1(4 - 16) = 1 - 0.1(-12) = 2.2$$

$$x_2 = 2.2 - 0.1(4(2.2)^3 - 16(2.2)) = 2.2 - 0.1(42.592 - 35.2) = 1.4608$$

$$x_3 = 1.4608 - 0.1(4(1.4608)^3 - 16(1.4608)) = 1.4608 - 0.1(12.467 - 23.373) = 1.4608 + 1.0906 = 2.5514$$

$\alpha = 0.5$:

$$x_1 = 1 - 0.5(-12) = 7$$

$$x_2 = 7 - 0.5(4(343) - 112) = 7 - 0.5(1372 - 112) = 7 - 630 = -623$$

$$x_3 = -623 - 0.5(4(-623)^3 - 16(-623)) \quad (\text{explodes})$$

(c) $\alpha = 0.1$ works better. $\alpha = 0.5$ causes divergence due to large gradients and high curvature.

(d) Critical points: $f'(x) = 4x(x^2 - 4) = 0 \Rightarrow x = 0, \pm 2$

6.3 Problem 3: Multi-variable Gradient Descent

Practice Problem

Minimize $f(x, y) = x^2 + 2y^2 + 2xy + 4x + 6y + 5$

(a) Find partial derivatives $\frac{\partial f}{\partial x}$ and $\frac{\partial f}{\partial y}$

(b) Start at $(x_0, y_0) = (0, 0)$, $\alpha = 0.1$, perform 2 iterations

(c) Find analytical minimum by solving $\nabla f = 0$

(d) Is the function convex? Why?

Solution:

(a) $\frac{\partial f}{\partial x} = 2x + 2y + 4, \frac{\partial f}{\partial y} = 4y + 2x + 6$

(b)

Iteration 1: $\nabla f(0, 0) = (4, 6)$

$$x_1 = 0 - 0.1 \times 4 = -0.4$$

$$y_1 = 0 - 0.1 \times 6 = -0.6$$

Iteration 2: $\nabla f(-0.4, -0.6) = (2(-0.4) + 2(-0.6) + 4, 4(-0.6) + 2(-0.4) + 6)$
 $= (-0.8 - 1.2 + 4, -2.4 - 0.8 + 6) = (2.0, 2.8)$

$$x_2 = -0.4 - 0.1 \times 2.0 = -0.6$$

$$y_2 = -0.6 - 0.1 \times 2.8 = -0.88$$

(c) Solve system:

$$2x + 2y + 4 = 0$$

$$2x + 4y + 6 = 0$$

Subtract: $2y + 2 = 0 \Rightarrow y = -1$

Substitute: $2x + 2(-1) + 4 = 0 \Rightarrow 2x + 2 = 0 \Rightarrow x = -1$

Minimum at $(-1, -1), f(-1, -1) = 2$

(d) Yes, convex because Hessian is positive definite:

$$H = \begin{bmatrix} 2 & 2 \\ 2 & 4 \end{bmatrix}, \quad \det(H) = 8 - 4 = 4 > 0, \quad \text{trace}(H) = 6 > 0$$

6.4 Problem 4: Linear Regression Gradient Descent

Practice Problem

Given data: $x = [1, 2], y = [3, 5]$, model: $\hat{y} = mx + c$

(a) Write MSE loss function $L(m, c)$

(b) Find partial derivatives $\frac{\partial L}{\partial m}$ and $\frac{\partial L}{\partial c}$

(c) Start with $m_0 = 0, c_0 = 0, \alpha = 0.1$, perform 3 iterations

(d) Find analytical solution using normal equations

Solution:

(a)

$$L(m, c) = \frac{1}{2}[(3 - (m + c))^2 + (5 - (2m + c))^2]$$

(b)

$$\frac{\partial L}{\partial m} = -(3 - m - c) - 2(5 - 2m - c) = -13 + 5m + 3c$$

$$\frac{\partial L}{\partial c} = -(3 - m - c) - (5 - 2m - c) = -8 + 3m + 2c$$

(c)

Iteration 1: $m_1 = 1.3, c_1 = 0.8$

Iteration 2: $m_2 = 1.71, c_2 = 1.05$

Iteration 3: $\nabla L(1.71, 1.05) = (-13 + 8.55 + 3.15, -8 + 5.13 + 2.10) = (-1.3, -0.77)$

$$m_3 = 1.71 - 0.1(-1.3) = 1.84$$

$$c_3 = 1.05 - 0.1(-0.77) = 1.127$$

(d) Normal equations:

$$\begin{bmatrix} 5 & 3 \\ 3 & 2 \end{bmatrix} \begin{bmatrix} m \\ c \end{bmatrix} = \begin{bmatrix} 13 \\ 8 \end{bmatrix}$$

Solve: $m = 2, c = 1$ (exact solution since data fits line perfectly)

7 Summary and Key Takeaways

7.1 Core Concepts Recap

Concept	Essential Understanding
Gradient Descent	Iterative optimization algorithm that follows negative gradient to find minima
Learning Rate (α)	Step size - critical hyperparameter that balances speed vs stability
Gradient (∇f)	Direction of steepest ascent - vector of partial derivatives
Loss Function	Measures prediction error - what we minimize in machine learning
Backpropagation	Chain rule applied to neural networks for efficient gradient computation
Momentum	Accumulates past gradients for smoother, faster convergence
Convexity	Property ensuring unique global minimum and guaranteed convergence
Stochastic GD	Uses random samples for faster updates on large datasets

7.2 Gradient Descent Variants Comparison

Variant	Advantages	Disadvantages
Batch GD	• Stable convergence • Deterministic	• Slow for large datasets • Memory intensive
Stochastic GD	• Fast updates • Escapes local minima	• High variance • May not converge
Mini-batch GD	• Balance of speed/stability • GPU efficient	• Batch size tuning • Slight variance
Momentum	• Faster convergence • Reduces oscillations	• Extra hyperparameter • May overshoot
Adam	• Adaptive learning rates • Combines momentum ideas	• More hyperparameters • Can generalize poorly

7.3 Implementation Guidelines

1. **Always normalize/standardize** input features to similar scales
2. **Initialize weights properly** using Xavier/Glorot or He initialization
3. **Monitor loss and metrics** during training with validation sets
4. **Use learning rate schedules** (step decay, exponential, cosine)
5. **Implement early stopping** based on validation performance
6. **Use regularization** (L1/L2, dropout) to prevent overfitting
7. **Experiment with optimizers:** Start with Adam, try SGD with momentum
8. **Visualize gradients** and learning curves for debugging
9. **Use gradient clipping** for stability in RNNs/Transformers
10. **Consider second-order methods** (L-BFGS) for small datasets

7.4 Common Pitfalls and Solutions

- **Vanishing gradients:** Use ReLU activations, skip connections, batch norm
- **Exploding gradients:** Gradient clipping, weight regularization
- **Local minima:** Multiple restarts, momentum, simulated annealing
- **Saddle points:** Second-order methods, adaptive learning rates
- **Poor conditioning:** Feature scaling, preconditioning, adaptive methods
- **Overfitting:** Regularization, dropout, early stopping, data augmentation
- **Underfitting:** Increase model capacity, reduce regularization, longer training
- **Oscillations:** Reduce learning rate, increase momentum, use adaptive LR

Appendix: Essential Formulas Reference

📖 Mathematical Foundation

Gradient Descent Update:

$$\theta_{t+1} = \theta_t - \alpha \nabla_{\theta} J(\theta_t)$$

Momentum Update:

$$\begin{aligned} v_t &= \gamma v_{t-1} + \alpha \nabla_{\theta} J(\theta_t) \\ \theta_{t+1} &= \theta_t - v_t \end{aligned}$$

Adam Update:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\ \theta_{t+1} &= \theta_t - \frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \end{aligned}$$

Mean Squared Error:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Cross-Entropy Loss:

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Chain Rule (Backpropagation):

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \cdot \frac{\partial z}{\partial w}$$

Sigmoid Derivative:

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

ReLU Derivative:

$$\text{ReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$$

Hessian Matrix (Second Derivatives):

$$H_{ij} = \frac{\partial^2 f}{\partial x_i \partial x_j}$$

Convexity Condition:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y), \quad \forall \lambda \in [0, 1]$$

Final Thoughts

Gradient Descent is the fundamental optimization algorithm powering modern machine learning and deep learning. While conceptually simple, its practical application requires careful consideration of learning rates, convergence criteria, and

problem-specific adaptations.

Remember:

- Mathematics provides the foundation, but experimentation is key
- There's no universal best optimizer - problem characteristics matter
- Monitoring and visualization are crucial for successful training
- Understanding the limitations helps choose appropriate solutions

Next Steps for Further Learning:

1. Implement gradient descent from scratch in Python/NumPy
2. Experiment with different learning rate schedules
3. Implement backpropagation for a 3-layer neural network
4. Study advanced optimizers (RMSprop, AdaGrad, Nadam)
5. Explore second-order optimization methods (Newton, BFGS)
6. Learn about optimization in constrained problems
7. Study convergence theory and rates
8. Explore distributed optimization for large-scale problems

"In theory, theory and practice are the same. In practice, they are not."
- Albert Einstein (paraphrased)
